

Telegram Desktop: Проблемы сборки и решения (V2)

Проект: @AndranikFutureLabs

В данном документе описаны технические сложности, возникшие при сборке Telegram Desktop в специфическом окружении (Windows Server 2025, Visual Studio 2026 Insiders), и методы их решения.

1. Используемые команды

Подготовка окружения (Native Tools Command Prompt x64):

```
:: Установка переменных окружения Visual Studio
call "C:\Program Files\Microsoft Visual Studio\18\Insiders\VC\Auxiliary\Build\vcvars64.bat"
```

```
:: Клонирование репозитория (если не сделано)
git clone --recursive https://github.com/telegramdesktop/tdesktop.git
cd tdesktop
```

Сборка зависимостей:

```
:: Запуск скрипта подготовки с указанием версии Qt
python Telegram/build/prepare/prepare.py qt6
```

Генерация проекта и сборка:

```
:: Генерация файлов проекта через CMake
cmake -B out -G "Visual Studio 18 2026" -A x64 -DCMAKE_BUILD_TYPE=Debug
```

```
:: Сборка цели Telegram
cmake --build out --config Debug --target Telegram
```

2. Модификация модуля prepare.py

Для успешной сборки на Windows Server 2025 с использованием Visual Studio 2026 (v18) были внесены критические правки в скрипт Telegram/build/prepare/prepare.py.

Исправление кодировки (UTF-8)

Для предотвращения ошибок UnicodeDecodeError при чтении вывода системных команд:

```
# Строка ~1934
currentCodePage = subprocess.run('chcp', capture_output=True, shell=True, text=True, env=modifiedEnv).stdout.strip().split()[-1]
subprocess.run('chcp 65001 > nul', shell=True, env=modifiedEnv)
runStages()
subprocess.run('chcp ' + currentCodePage + ' > nul', shell=True, env=modifiedEnv)
```

Автозамена Toolset (v143) и версии VS

Скрипт был дополнен логикой автоматической подмены устаревших версий тулсета на актуальный для VS 2026:

```
# Строка ~433
commands = commands.replace('v140', 'v143').replace('v141', 'v143').replace('v142', 'v143')
commands = commands.replace('Visual Studio 17 2022', 'Visual Studio 18 2026')
```

Патчинг Breakpad

Модуль breakpad требовал особого внимания из-за жестко прописанных путей и версий в .vsxproj файлах:

Строка ~438

```
if stage['name'] == 'breakpad':
```

```
    patch_cmd = 'python -c "import glob; [open(f, \'wb\').write(open(f, \'rb\').read().replace(b\'v140\', b\'v143\').replace(b\'v141\', b\'v143\').replace(b\'v142\'
```

```
    # ... замена команд msbuild с внедрением patch_cmd
```

3. Решение ошибок MSBuild

Ошибки MSB1011, MSB4025, MSB4102

Эти ошибки возникали из-за неправильного определения путей к таргетам и версии SDK в новой версии Visual Studio.

Решение: В скрипте prepare.py была реализована принудительная передача параметров VCTargetsPath и PlatformToolset во все вызовы MSBuild:

Строка ~430

```
msbuild_path = "C:\\Program Files\\Microsoft Visual Studio\\18\\Insiders\\MSBuild\\Current\\Bin\\amd64\\MSBuild.exe"
```

```
vc_targets = "C:\\Program Files\\Microsoft Visual Studio\\18\\Insiders\\MSBuild\\Microsoft\\VC\\v170"
```

Строка ~450

```
commands = commands.replace('msbuild', f'{msbuild_path} /p:PlatformToolset=v143 /p:WindowsTargetPlatformVersion=10.0 /p:VCTargetsPath="{vc_t
```

Это позволило MSBuild корректно находить файлы Microsoft.Cpp.Default.props и другие компоненты системы сборки, которые в Insiders-версии VS находятся по нестандартным путям.

Документация подготовлена @AndranikFutureLabs. Версия V2.